

PROGETTO INFRASTRUTTURA DI RETE THETA



PPSMT_{inc.}

INDICE

1 Preventivo Infrastruttura di Rete	3
1.1 Materiali	
1.2 Figure necessarie e tempistiche	
1.3 Eventuali licenze Software	
1.4 Costi opzionali di manutenzione annuale	
2 Topologia di Rete e configurazione	6
2.1 Visualizzazione e descrizione della Topologia	
2.2 Configurazione di Rete	
2.3 Topologia descrittiva e funzionamento della rete	
3 Virtualizzazione della Rete	12
3.1 Introduzione	
3.2 Configurazione delle schede di Rete	
3.3 Firewall Pfsense	
3.4 Regole Firewall	
4 Strumenti di analisi Python	17
4.1 Port scanner	
4.2 Analizzatore Verbi HTTP	
5 Testing e Ping	20
5.1 Ping PC - Web Server	
5.2 Verifica verbi HTTP	
5.3 Port Scanning	
6 Bonus	22
6.1 Subnetting	
6.2 Socket Sniffer	
7 Conclusioni	25
7.1 Suddivisione del Lavoro	
7.2 Considerazioni finali	

1 Preventivo infrastruttura di rete

Azienda Theta



1.1 Materiali

L'obiettivo è creare un'infrastruttura di rete aziendale in un edificio di 6 piani con i seguenti materiali forniti dall'azienda.

Materiale	Quantità	
PC	120	Dell OptiPlex 3080 SFF
Switch	6	Cisco Catalyst serie 2960
Firewall	1	Cisco ASA 5506
IDS	3	Juniper idp250
Server	1	Dell PowerEdge T150
NAS	1	Synology DS925
Router	1	Cisco 2911

Oltre a ciò l'azienda mette a disposizione anche cavi Ethernet, canaline, rack e tutto ciò che occorre per configurare la rete.

1.2 Figure necessarie e tempistiche

In questa sezione vedremo tutto ciò che riguarderà il nostro lavoro nel pratico, i giorni necessari di presenza per ogni ruolo, il costo parziale delle mansioni.

Questa infrastruttura di rete ha bisogno di figure specializzate, nello specifico.

- **Network engineer:** sarà sostanzialmente il capoprogetto che supervisionerà tutto il lavoro e si occuperà della progettazione della rete e di tutte le configurazioni a livello software e hardware dei vari componenti.
- **IT technician:** si occuperà dell'installazione sulle macchine di tutti i software necessari come antivirus, sistemi operativi etc. Inoltre si occuperà di eseguire i ping e i test per capire se tutti i componenti sono stati configurati correttamente.

- **Tecnico Cablatore:** è la figura che si occuperà della stesura dell'impianto di cablaggio che collegherà tutti i componenti, inoltre eseguirà il cable management di tutta l'infrastruttura.

Detto ciò vediamo i costi e i giorni di presenza di ogni figura:

Ruolo	Tempistiche	Lavoro	Prezzo
1 Network engineer/ Project Manager	12 Giorni	Capoprogetto, progettazione VLAN, config. router/firewall/IDS/Switch/Server/NAS	500€/g IVA inc.
2 IT technician	10 Giorni	Installazione SO su PC/server, config antivirus, testing ping/tracciati/bandwidth	250€/g IVA inc.
3 Tecnici Cablatore	10 Giorni	Stesura cavi Cat6, canaline, rack e cable management	300€/g IVA inc.

1.3 Eventuali Licenze Software

Per operare, in caso l'azienda non ne disponga, Theta avrà bisogno delle licenze per i sistemi operativi, antivirus, pacchetti Office, Server e Firewall.

Licenza	Quantità	Prezzo stimato
Windows 11 Pro	120	120x50€ = 6000€ Acquisto singolo
Windows Server	1	1000€/anno
Microsoft 365 Business	120	120x80€/anno= 9.600€
Firewall Advanced Protection Service Suite	1	1.100€/anno
Antivirus Webroot	120	120x25€/anno= 3000€
Juniper SRX + Advanced Threat Prevention	3	3x600€ = 1800€/anno

1.4 Costi opzionali di manutenzione annuale

Oltre alle licenze software e l'installazione effettiva dell'infrastruttura, ci sono dei costi di manutenzione che non vanno trascurati per mantenerla funzionante nel tempo. La loro assenza potrebbe generare costi imprevisti o, più criticamente, rischi operativi e di sicurezza.

Voce di Manutenzione	Descrizione	Prezzo stimato/anno
Backup & Disaster Recovery	Soluzione cloud/software (es. Veeam, Acronis)	500€ - 1.500€
Network Monitoring	PRTG, Zabbix o simili per monitoraggio H24	500€ - 1.200€
Connettività ISP	Linea Internet business dedicata (fibra)	1.200€ - 3.600€
Supporto IT continuativo	Helpdesk post-installazione per 120 utenti	3.000€ - 8.000€
Manutenzione UPS	Sostituzione batterie gruppi di continuità	200€ - 500€
VPN Aziendale	Licenza VPN per accesso remoto dipendenti	300€ - 800€

1.5 Riepilogo e Totale

Tempo stimato di
installazione

12 giorni

Licenze Software

16.500€/anno
+ **6000€** IVA inc.

Costi totali Installazione

11.500€ IVA inc.

Costi di manutenzione
opzionali

6000 -
15.600€/anno IVA inc.

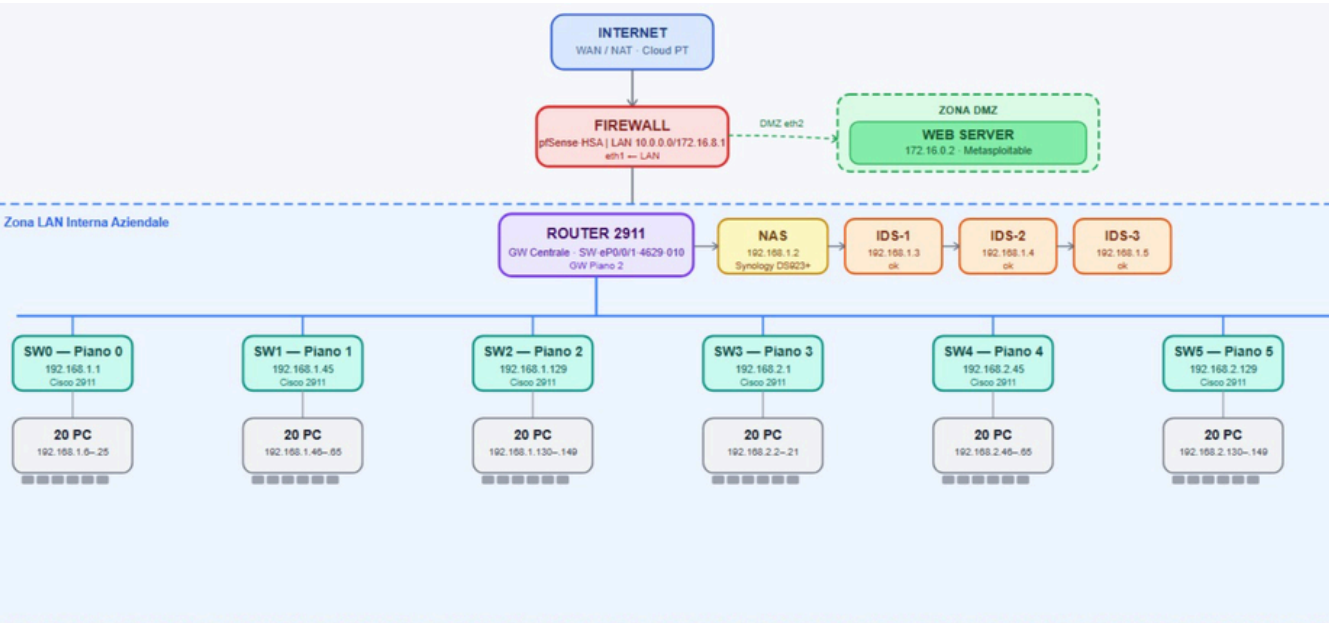
2 Topologia di rete e configurazione

Azienda Theta



La topologia di rete ci serve a visualizzare, piano per piano, tutte le disposizioni delle macchine, come sono disposti i rack, e come sono configurati a livello di rete i dispositivi forniti.

2.1 Visualizzazione e descrizione della topologia



La topologia è suddivisa in zone distinte con l’obiettivo di aumentare la sicurezza interna e verso l’esterno, abbiamo quindi:

- Zona WAN verso internet
- Zona DMZ per i servizi esposti pubblicamente
- Zona LAN per la rete interna aziendale

Componenti principali dell’ infrastruttura

Materiale	Quantità	Modello
PC	120	Dell OptiPlex 3080 SFF
Switch	6	Cisco Catalyst serie 2960
Firewall	1	Cisco ASA 5506

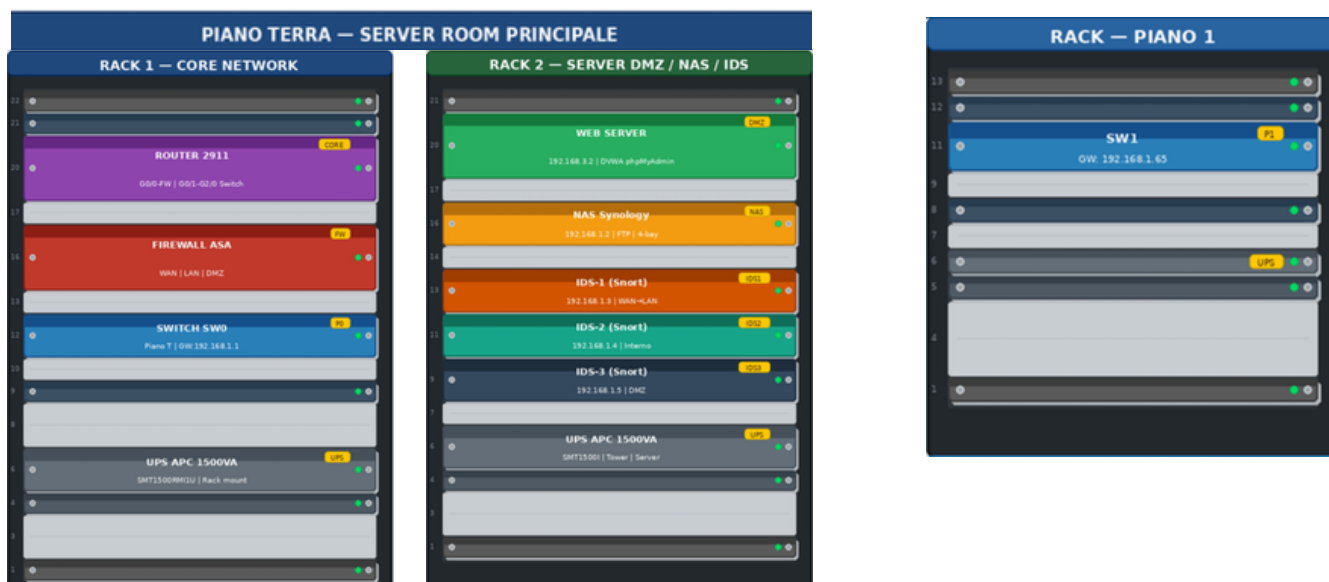
Materiale	Quantità	Modello
IDS	3	Juniper idp250
Server	1	Dell PoverEdge T150
NAS	1	Synology DS925
Router	1	Cisco 2911

Suddivisione dei dispositivi per piano

Piano	Dispositivi
Piano Terra	20 PC + Router + Switch 0 + NAS + IDS (1,2,3) + Web Server
Piano 1	20 pc + Switch 1
Piano 2	20 pc + Switch 2
Piano 3	20 pc + Switch 3
Piano 4	20 pc + Switch 4
Piano 5	20 pc + Switch 5

Disposizione Rack

Abbiamo anche creato uno schema di impostazione delle rack piano per piano per visualizzare meglio il lavoro da svolgere e avere una panoramica più completa.





Come descritto dalla immagini abbiamo deciso di creare una Server Room principale al piano terra per avere una soluzione più centralizzata possibile.

2.2 Configurazione di rete

In questa sezione andremo a vedere nel pratico come abbiamo deciso di configurare ogni dispositivo. Per efficienza dato il numero dei PC, la configurazione IP è stata fatta in DHCP tramite il router.

Configurazione PC

Piano	Gateway	Range Dispositivi	Subnet mask
Piano Terra	192.168.1.1	192.168.1.6-25	255.255.255.192
Piano 1	192.168.1.65	192.168.1.66-85	255.255.255.192
Piano 2	192.168.1.129	192.168.1.130-149	255.255.255.192
Piano 3	192.168.2.1	192.168.2.2-21	255.255.255.192
Piano 4	192.168.2.65	192.168.2.66-85	255.255.255.192
Piano 5	192.168.2.129	192.168.2.130-149	255.255.255.192

Configurazione Firewall

Interfaccia di rete	Gateway	Subnet Mask	Tipologia Rete	Collegamento
em0	200.0.0.1	255.255.255.0	WAN	Internet
em1	10.0.0.1	255.255.255.0	LAN	Router
em2	172.16.0.1	255.255.255.0	DMZ	Web Server

Configurazione Router

Interfaccia di rete	IP Address	Subnet Mask	Dispositivi
eth0	192.168.1.1	255.255.255.192	SW0
eth1	192.168.1.65	255.255.255.192	SW1
eth2	192.168.1.129	255.255.255.192	SW2
eth3	192.168.2.1	255.255.255.192	SW3
eth4	192.168.2.65	255.255.255.192	SW4
eth5	192.168.2.129	255.255.255.192	SW5
eth6	192.168.1.1	255.255.255.192	Firewall
eth7	192.168.1.1	255.255.255.192	NAS
eth8	192.168.1.1	255.255.255.192	IDS-1
eth9	192.168.1.1	255.255.255.192	IDS-2
eth10	192.168.1.1	255.255.255.192	IDS-3

Nel diagramma gli IDS sono rappresentati come collegati al router per semplicità grafica, ma nella realtà operano tramite port mirroring (SPAN) sugli switch per analizzare il traffico.

Configurazione IDS

IDS	Gateway	Ip	Subnet Mask
IDS1	192.168.1.1	192.168.1.3	255.255.255.192
IDS2	192.168.1.1	192.168.1.4	255.255.255.192
IDS3	192.168.1.1	192.168.1.5	255.255.255.192

Configurazione NAS

Gateway	Ip	Subnet Mask
192.168.1.1	192.168.1.2	255.255.255.192

2.3 Topologia Descrittiva e Funzionamento della rete

La rete è organizzata in modo gerarchico: ogni comunicazione parte sempre dal PC, sale allo Switch del piano, poi al Router, e da lì al Firewall se deve uscire dalla LAN. Questa gerarchia non è casuale ma serve a separare i compiti: lo Switch collega i dispositivi dello stesso piano, il Router instrada il traffico tra i piani, il Firewall controlla e filtra tutto ciò che entra ed esce dalla rete aziendale.

PC naviga su Internet

Il PC manda il pacchetto allo Switch del suo piano, che lo passa al Router. Il Router non riconosce la destinazione come locale e lo manda al Firewall seguendo la route di default. Il Firewall controlla le regole, autorizza il traffico, applica il NAT sostituendo l'IP privato con il proprio IP pubblico e lo spedisce verso Internet. La risposta torna seguendo il percorso inverso. Il NAT è stato configurato perché gli indirizzi privati come 192.168.x.x non sono instradabili su Internet. Mascherando l'IP del PC con l'IP pubblico del Firewall si ottiene anche un beneficio di sicurezza: dall'esterno è impossibile conoscere la struttura interna della rete.

PC accede al NAS

Il PC manda il pacchetto allo Switch, che lo passa al Router. Il Router riconosce che la destinazione appartiene alla subnet del Piano Terra e lo instrada direttamente verso lo Switch0, che lo consegna al NAS. Questo traffico non passa mai per il Firewall. Il NAS è stato volutamente escluso dal controllo del Firewall per il traffico interno perché sarebbe un collo di bottiglia inutile: i 120 PC trasferiscono file continuamente e far passare tutto quel traffico per il Firewall rallenterebbe la rete senza aggiungere sicurezza, dato che il NAS è già protetto all'interno della LAN e non è raggiungibile dall'esterno.

Internet accede al Web Server

Il pacchetto arriva sull'interfaccia WAN del Firewall. Il Firewall trova la regola di NAT Port Forwarding sulla porta 80 e redirige il traffico verso il Web Server nella DMZ. I PC interni non vedono mai questo traffico.

Il Web Server è stato posizionato nella DMZ e non nella LAN proprio per questo motivo: essendo accessibile da Internet è un bersaglio esposto. Se fosse nella LAN e venisse compromesso, l'attaccante si troverebbe direttamente nella stessa rete dei 120 PC aziendali. La DMZ crea un segmento isolato dove il server può operare senza mettere a rischio la rete interna.

Il Web Server viene attaccato

Se un attaccante compromette il Web Server e tenta di raggiungere i PC interni, il Firewall blocca immediatamente ogni pacchetto che parte dalla DMZ verso la LAN. L'attaccante rimane intrappolato nella DMZ.

Questa regola di blocco è la più importante dell'intera configurazione. Senza di essa la DMZ sarebbe inutile: un attaccante che compromette il Web Server potrebbe muoversi liberamente verso i PC interni, il NAS con i dati aziendali e tutti gli altri dispositivi. Con questa regola invece la compromissione del Web Server non si propaga mai alla rete interna.

Il DHCP

Quando un PC si accende manda un broadcast DHCP sulla rete. Lo Switch lo passa al Router, che riconosce da quale interfaccia è arrivato, capisce a quale piano appartiene il PC e risponde con un indirizzo IP del pool corrispondente, la subnet mask, il gateway e il DNS. Il DHCP è stato configurato direttamente sul Router invece che su un server dedicato perché il Router già conosce tutte le subnet dei sei piani e può gestire i pool in modo centralizzato. Questo semplifica la gestione: aggiungere un nuovo PC su qualsiasi piano non richiede nessuna configurazione manuale, il dispositivo riceve automaticamente tutti i parametri corretti appena si accende.

Gli IDS

IDS-1 monitora il traffico in entrata da Internet cercando attacchi esterni che hanno superato il firewall.

IDS-2 monitora il traffico interno tra i piani per rilevare eventuali malware in propagazione.

IDS-3 monitora il traffico verso il Web Server nella DMZ per individuare attacchi ai servizi esposti.

Gli IDS operano in modalità passiva e ricevono una copia del traffico tramite port mirroring (SPAN) configurato sugli apparati di rete, permettendo di analizzare i flussi senza interferire con il traffico reale.

Sono stati scelti tre IDS separati invece di uno solo perché ognuno può essere specializzato con regole diverse in base al tipo di traffico monitorato. Un singolo IDS dovrebbe gestire tutto il traffico contemporaneamente, diventando un possibile collo di bottiglia; inoltre, in caso di guasto, l'intero sistema di monitoraggio verrebbe compromesso. Utilizzando tre sistemi distinti, invece, un eventuale malfunzionamento impatta solo una parte della rete, mantenendo attivo il resto del sistema di analisi.

3 Virtualizzazione della Rete

Azienda Theta



3.1 Introduzione

Dopo aver definito la topologia e il preventivo, abbiamo prima costruito e configurato l'infrastruttura di rete virtuale e poi eseguito una fase di testing sull'ambiente simulato. Il lavoro si è sviluppato in tre momenti distinti e dipendenti l'uno dall'altro: la preparazione delle schede di rete in VirtualBox, la configurazione del firewall tramite pfSense, e infine l'analisi della sicurezza tramite strumenti scritti in Python. (Vedi Sezione 5. Ping e Testing)

Suddivisione della rete

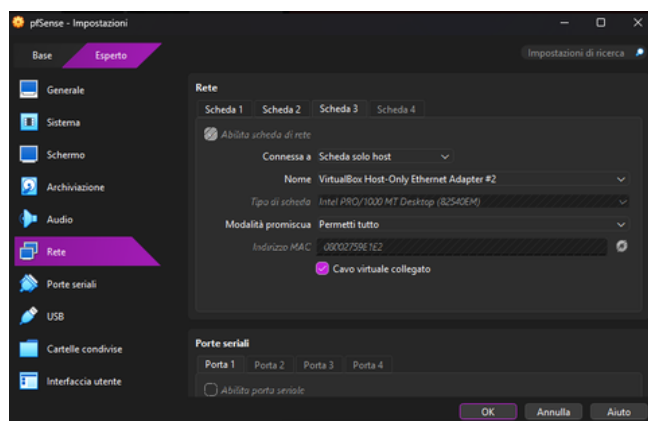
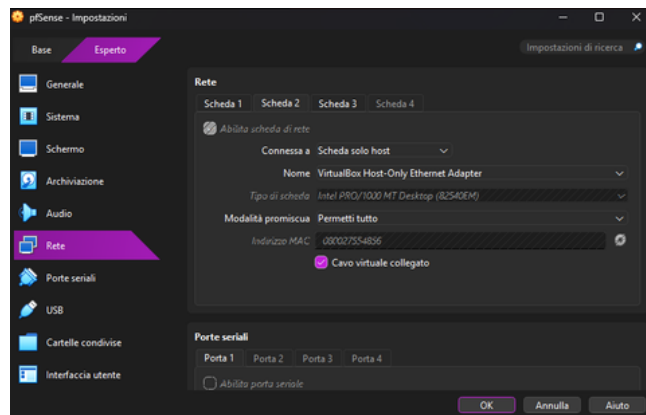
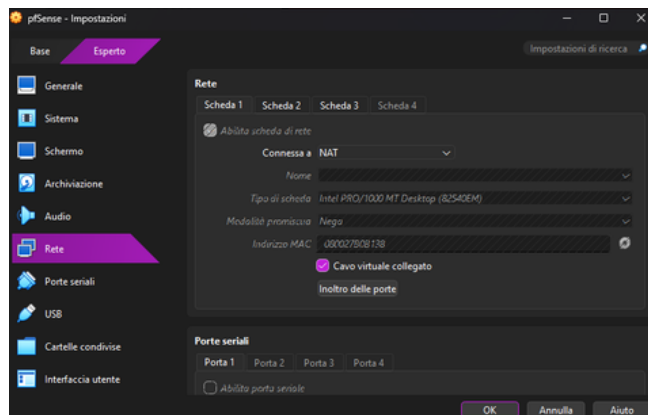
La topologia del progetto è divisa in tre zone con tre ruoli distinti. La WAN è la connessione verso internet, quindi la parte più esposta verso dell'intera infrastruttura, la DMZ è la zona dove si trova il web server aziendale: è raggiungibile dall'esterno ma è separata dalla rete interna in modo da proteggere i computer da un eventuale attacco al server, infine abbiamo la LAN che è la rete interna aziendale, la zona più protetta, dove operano i 120 PC distribuiti sui 6 piani dell'edificio. Ogni zona è separata fisicamente dalle altre attraverso interfacce diverse del firewall, e nessun traffico può passare da una zona all'altra senza transitare per pfSense. Nella simulazione, il web server aziendale è rappresentato dalla macchina Metasploitable all'indirizzo 172.16.0.2 nella DMZ, mentre Kali Linux rappresenta un PC operativo nella rete LAN interna. pfSense gestisce il confine tra le due reti e verso l'esterno.

3.2 Configurazione delle schede di Rete

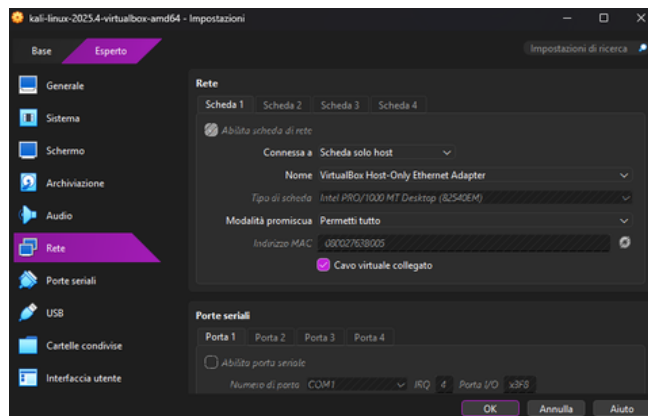
Logica della configurazione

In VirtualBox esistono diverse modalità di connessione per le schede di rete, ognuna con un comportamento diverso rispetto alla rete dell'host e alle altre macchine virtuali. La modalità NAT, usata per la prima scheda di pfSense, permette alla macchina virtuale di accedere a internet passando attraverso la connessione del computer host, senza essere direttamente raggiungibile dall'esterno. È la scelta migliore per simulare una connessione WAN: pfSense riceve un indirizzo IP da VirtualBox tramite DHCP, esattamente come farebbe un router collegato a un provider internet. La modalità Scheda solo host crea invece una rete completamente isolata che esiste solo tra le macchine virtuali collegate allo stesso adattatore. Non c'è accesso diretto a internet e nessuna comunicazione con la rete fisica del computer host. Questa è la scelta corretta per i segmenti interni perché garantisce che il traffico rimanga confinato all'interno del laboratorio virtuale, replicando fedelmente l'isolamento tra LAN e DMZ. Ora vediamo nel dettaglio la configurazione delle schede di rete:

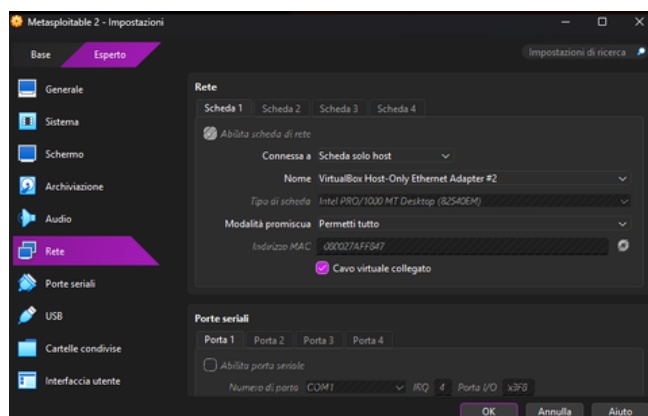
Pfsense



Kali



Metasploitable



Assegnazione delle schede per ogni macchina

pfSense è stata configurata con tre schede di rete. La prima è in modalità NAT e corrisponde all'interfaccia WAN. La seconda è collegata al primo adattatore host-only e corrisponde all'interfaccia LAN, il segmento dove si trova Kali Linux. La terza è collegata al secondo adattatore host-only e corrisponde all'interfaccia OPT1, ovvero la DMZ dove risiede Metasploitable.

Kali Linux è stata collegata al primo adattatore host-only, lo stesso della seconda scheda di pfSense, posizionandola nella LAN. Metasploitable è stata collegata al secondo adattatore host-only, lo stesso della terza scheda di pfSense, posizionandola nella DMZ. In questo modo le due macchine non si trovano mai sullo stesso segmento di rete e qualsiasi comunicazione tra loro deve necessariamente passare per pfSense.

3.3 Firewall Pfsense

Con le schede di rete configurate correttamente in VirtualBox, il passo successivo è stato avviare pfSense e completare la sua configurazione interna. PfSense è un sistema operativo open source basato su FreeBSD che trasforma una macchina, anche virtuale, in un firewall e router completamente configurabile. È stato scelto perché è lo strumento indicato nella traccia per simulare il firewall perimetrale della Compagnia Theta, e nella topologia rappresenta l'unico punto di ingresso e uscita tra le tre zone della rete.

Assegnazione delle interfacce e indirizzi IP

All'avvio, pfSense ha rilevato le tre schede di rete configurate in VirtualBox e ha chiesto di associarle alle interfacce logiche WAN, LAN e OPT1. Questo passaggio è il collegamento tra la configurazione fisica fatta in VirtualBox e la configurazione logica di pfSense: em0 è diventata la WAN, em1 la LAN e em2 l'OPT1. L'interfaccia WAN (em0) è stata configurata con il gateway 200.0.0.1, il punto di connessione verso Internet. La LAN (em1) è stata configurata con l'indirizzo statico 10.0.0.1 con subnet mask 255.255.255.0 che funge da gateway verso il router interno. La DMZ (em2) è stata configurata con l'indirizzo statico 172.16.0.1 con la stessa subnet mask, che funge da gateway per Metasploitable, il web server aziendale simulato.

Usare indirizzi statici su LAN e DMZ è una scelta obbligata per un firewall: se il gateway cambiasse indirizzo le macchine collegate non saprebbero più dove instradare il traffico, rendendo la rete inutilizzabile.

Regole di traffico e ruolo della DMZ

Il principio fondamentale della configurazione pfSense è che tutto il traffico deve passare per il firewall, senza eccezioni. La regola più importante riguarda il traffico dalla DMZ verso la LAN, che è configurato come bloccato in modo permanente. Questo significa che anche se un attaccante riuscisse a compromettere il web server nella DMZ, non potrebbe usarlo come punto di partenza per attaccare i PC interni: il firewall fermerebbe qualsiasi tentativo di connessione dalla DMZ verso la LAN.

Il traffico dalla LAN verso la DMZ è invece permesso sulla porta 80, il che consente a Kali Linux di raggiungere il web server Metasploitable.

Questo rispecchia uno scenario realistico in cui i PC aziendali possono consultare il web server interno, ma il web server non può mai iniziare connessioni verso i PC. Il traffico dalla LAN verso Internet è permesso con NAT: pfSense sostituisce l'IP interno con il proprio IP WAN, nascondendo la struttura interna della rete verso l'esterno.

Verifica del funzionamento

A configurazione completata, l'accesso alla dashboard web di pfSense dall'indirizzo 10.0.0.1 ha confermato che tutte e tre le interfacce erano attive con gli indirizzi corretti. Prima di procedere con i test abbiamo verificato che Kali Linux riuscisse a raggiungere Metasploitable attraverso il firewall, condizione necessaria per poter eseguire le analisi successive.

Ifconfig PfSense

```
WAN (wan)      -> em0      -> v4/DHCP4: 10.0.2.15/24
                v6/DHCP6: fd17:625c:f037:2:a00:27ff:feb0:b138/
v4
LAN (lan)      -> em1      -> v4: 10.0.0.1/24
OPT1 (opt1)    -> em2      -> v4: 172.16.0.1/24
```

3.4 Regole Firewall

Abbiamo immaginato che l'azienda Theta ci desse alcune regole da inserire nel firewall come il blocco verso alcuni siti in maniera tale che i dipendenti non potessero accedervi durante l'orario lavorativo.

Per farlo abbiamo disposto un blocco DNS verso alcuni siti. il DNS si occupa di tradurre in indirizzo ip ciò che scriviamo nella barra di ricerca, se blocchiamo il DNS quindi il sito diventa inaccessibile.

Regole

Il pacchetto che abbiamo installato per rendere possibile la creazione di liste contenenti i siti da bloccare si chiama pfBlockerNG-devel. Il funzionamento si basa facendo intervenire il firewall come DNS, ogni qual volta un client cercherà un sito che si trova dentro la black-list pfBlockerNG interviene nel processo di DNS Resolver e restituisce una VIP (Virtual IP) interna al firewall. Quindi il browser del client cercherà di stabilire una connessione con la VIP piuttosto che con il server del fornitore (in questo caso, Pornhub e Meta).

Firewall / pfBlockerNG / DNSBL / DNSBL Groups

GeneralIPDNSBLUpdateReportsFeedsLogsSync

DNSBL GroupsDNSBL CategoryDNSBL SafeSearch

DNSBL Groups Summary (Drag to change order)

Name	Description	Action	Frequency	Logging/Blocking Mode	
ADs_Basic	ADs Basic - Col...	Unbound ▾	Once a day ▾	DNSBL WebServer/VIP ▾	
EasyList	EasyList Feeds ...	Unbound ▾	Once a day ▾	DNSBL WebServer/VIP ▾	
SocialBlock		Unbound ▾	Once a day ▾	DNSBL WebServer/VIP ▾	

+ Add
Save

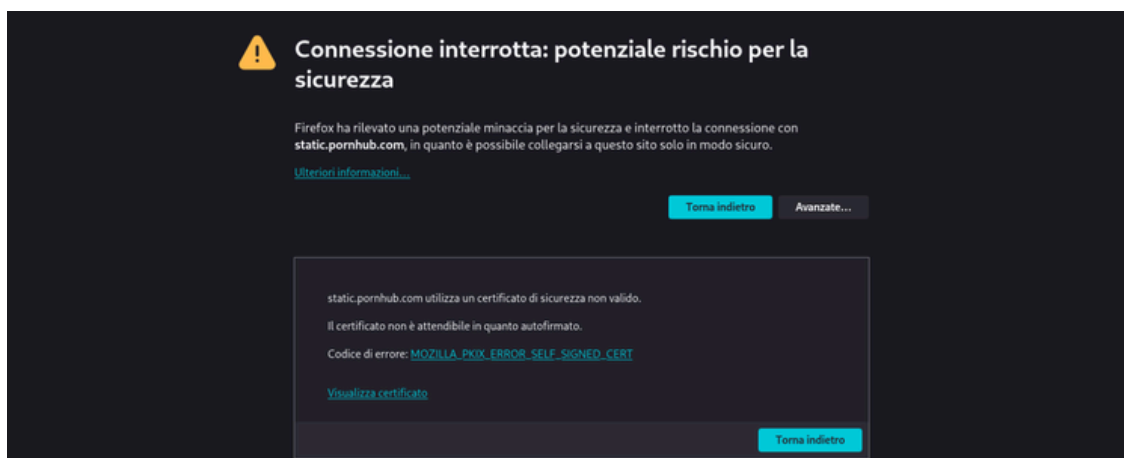
DNSBL Custom_List

No Regex Entries Allowed!

facebook.com
instagram.com
fb.com
fbcdn.com
messenger.com
pornhub.com
it.pornhub.com
youporn.com

Funzionamento

Poiché il sito richiesto è in HTTPS, il browser si aspetta un certificato validato da una CA pubblica per quel dominio. Ricevendo invece la risposta dalla VIP di pfBlockerNG (che usa un certificato autofirmato), il browser blocca la connessione per prevenire potenziali attacchi Man-in-the-Middle. Questo garantisce l'impossibilità di navigazione sul sito proibito



Per evitare bypass comune delle regole abbiamo abilitato la funzione TLD (Top Level Domain) che estende il blocco a tutti i sottodomini del blocco primario. Se si cercherà per esempio static.facebook.com verrà comunque bloccato.

4 Strumenti di analisi Python

Azienda Theta



Dopo aver completato tutta l'infrastruttura, la fase successiva è stata la realizzazione degli strumenti di analisi della rete. La traccia richiedeva di scrivere questi strumenti da zero in Python.

I tre programmi realizzati operano a livelli diversi della rete e rispondono a domande diverse sull'infrastruttura. Lo scanner di porte analizza quali servizi sono esposti sulla macchina bersaglio. Il verificatore di metodi HTTP approfondisce uno di questi servizi dal punto di vista del protocollo applicativo. Il packet sniffer osserva il traffico in modo passivo, senza interagire direttamente con nessun host. Insieme coprono una ricognizione completa dell'ambiente simulato.

4.1 Port Scanner

Con la rete operativa, abbiamo avviato la fase di testing. Il primo programma esegue la scansione delle porte del web server nella DMZ, operando da Kali Linux nella LAN attraverso il firewall. L'obiettivo era ottenere un quadro preciso dei servizi esposti da Metasploitable, che nella topologia rappresenta il web server aziendale: sapere quali porte sono aperte è il punto di partenza per valutare la superficie di attacco dell'intera infrastruttura.

Scelta degli strumenti

Per questo programma abbiamo usato le librerie socket e datetime, entrambe già incluse nell'installazione Python. L'unico modo per interagire con le porte di una macchina remota a questo livello era lavorare direttamente con le connessioni TCP. La libreria datetime è stata aggiunta per documentare l'orario della scansione nell'output.

Socket è una libreria che permette di creare e gestire connessioni di rete direttamente dal codice, mettendo a disposizione tutti gli strumenti necessari per comunicare con altri dispositivi attraverso i protocolli di rete standard come TCP e UDP. Datetime invece è una libreria più semplice, che serve a lavorare con date e orari, nel nostro caso è stata usata esclusivamente per registrare e stampare l'ora di inizio e fine della scansione, rendendo l'output più leggibile e utile per documentare quando è stato eseguito il test.

Approccio adottato e motivazioni

La logica del programma si basa sul tentare una connessione TCP su ogni porta nel range indicato e osservare il risultato. Questo approccio è il più diretto disponibile senza privilegi di amministratore: invece di inviare pacchetti grezzi e analizzare le risposte a basso livello, si sfrutta il meccanismo standard di connessione TCP del sistema operativo.

Impostare un limite di tempo per ogni tentativo era indispensabile per rendere il programma utilizzabile in pratica. Senza timeout, ogni porta chiusa o filtrata dal firewall avrebbe bloccato il programma in attesa indefinita, rendendo impraticabile la scansione di un range ampio. Abbiamo incluso anche il riconoscimento del nome del servizio associato a ogni porta aperta.

Codice Port Scanner

```
from datetime import datetime
import socket

target=input("Inserisci l'indirizzo IP da scansionare: ")
port_range=input("Inserisci il range di porte da scansionare (es. da 5-200): ")

porta_iniziale=int(port_range.split('-')[0])
porta_finale=int(port_range.split('-')[1])

print("\n" + "="*60)
print(f"SCANSIONE SU: {target}")
print(f"INIZIO: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
print("="*60)
print(f"{'PORTA':<10} {'STATO':<15} {'SERVIZIO':<20}")
print("="*60)

for porta in range (porta_iniziale, porta_finale + 1):
    s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.5)

    status=s.connect_ex((target,porta))

    if (status==0):
        try:
            servizio = socket.getservbyport(porta, "tcp")
        except:
            servizio = "Sconosciuto"

        print(f"{'porta':<10} {'[ APERTA ]':<15} {'servizio':<20}")
    else:
        print(f"{'porta':<10} {'[ CHIUSA ]':<15} {'-':<20}")
    s.close()

print("="*60)
print(f"FINE SCANSIONE: {datetime.now().strftime('%H:%M:%S')}")
print("="*60)
```

4.2 Analizzatore verbi HTTP

Il secondo programma si concentra sul servizio HTTP sulla porta 80. In particolare abbiamo puntato al pannello phpMyAdmin, accessibile al percorso (/phpMyAdmin/index.php), che è lo strumento di amministrazione del database MySQL. L'obiettivo era verificare quali metodi HTTP vengono accettati su quella risorsa: nella topologia questo è uno dei punti di accesso web più critici del server, e un pannello di amministrazione database mal configurato può rappresentare un vettore di attacco significativo.

Scelta degli strumenti

Per questo programma abbiamo scelto la libreria requests al posto di socket. La differenza è sostanziale: lavorare con HTTP richiede di gestire intestazioni, codici di risposta, redirect e molti altri dettagli del protocollo che con socket avremmo dovuto costruire manualmente come stringhe di testo. La libreria requests gestisce tutto questo in automatico, permettendoci di concentrarci sulla logica del programma, ovvero testare sistematicamente ogni metodo e classificare le risposte. Usare socket per HTTP avrebbe complicato inutilmente il codice senza portare nessun vantaggio reale per questo specifico obiettivo.

Approccio adottato e motivazioni

Il programma scorre una lista di metodi HTTP e per ciascuno invia una richiesta al percorso indicato, analizzando il codice di stato ricevuto. La classificazione delle risposte segue la logica del protocollo: un codice sotto 400 indica che il server ha elaborato la richiesta, un 401 o 403 indica che il metodo è riconosciuto ma l'accesso è negato, un 405 invece indica che il metodo non è ammesso su quella risorsa.

Abbiamo gestito anche i casi di errore di rete, poiché in un contesto di analisi è importante distinguere tra un server che risponde negativamente e un server che non risponde affatto. I due scenari hanno implicazioni diverse: nel primo caso il servizio è attivo ma configurato restrittivamente, nel secondo potrebbe esserci un problema di connettività o il servizio potrebbe essere filtrato dal firewall.

Codice Analizzatore verbi HTTP

```
GNU nano 8.7.1 http_verb.py
import requests
from datetime import datetime

def http_verbs(url, path):
    base_url = url.rstrip("/")
    target_path = "/" + path.lstrip("/")
    full_url = base_url + target_path
    verbs = ['GET', 'POST', 'PUT', 'DELETE', 'PATCH', 'HEAD', 'OPTIONS', 'TRACE', 'CONNECT']
    print("\n" + "="*50)
    print("ANALISI SICUREZZA HTTP")
    print(f"URL BERSAGLIO: {full_url}")
    print(f"DATA E ORA: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
    print("="*50 + "\n")
    abilitati = []
    protetti = []
    disabilitati = []

    for verb in verbs:
        try:
            r = requests.request(verb, full_url, timeout=5, allow_redirects=False)
            if r.status_code < 400:
                print(f"[+] {verb:8} : ABILITATO (Status: {r.status_code})")
                abilitati.append(verb)
            elif r.status_code in [401, 403]:
                print(f"[!] {verb:8} : PROTETTO/NEGATO (Status: {r.status_code})")
                protetti.append(verb)
            elif r.status_code == 405:
                print(f"[-] {verb:8} : NON AMMESSO (Status: 405)")
                disabilitati.append(verb)
            else:
                print(f"[-] {verb:8} : ERRORE/DISABILITATO (Status: {r.status_code})")
                disabilitati.append(verb)
        except requests.exceptions.ConnectionError:
            print(f"[X] {verb:8} : ERRORE DI CONNESSIONE")
        except requests.exceptions.Timeout:
            print(f"[X] {verb:8} : TIMEOUT (Server troppo lento)")
        except Exception as e:
            print(f"[X] {verb:8} : ERRORE INASPETTATO: {e}")

    print("\n" + "="*50)
    print("RIEPILOGO FINALE:")
    print(f"VERBI APERTI: {abilitati}")
    print(f"VERBI PROTETTI: {protetti}")
    print(f"VERBI CHIUSI: {disabilitati}")
    print("="*50 + "\n")

if __name__ == "__main__":
    target_url = input("Inserisci l'URL (es. http://example.com): ")
    target_path = input("Inserisci il percorso (es. /api/data): ")
    http_verbs(target_url, target_path)
```

5 Testing e Ping

Azienda Theta



Abbiamo ora impostato tutta l'infrastruttura di rete, l'abbiamo collegata al firewall pfsense, abbiamo definito le regole di passaggio dei pacchetti verso e dall'internet.

L'ultima cosa da fare è verificare che tutto funzioni tramite dei test.

Questi test consisteranno in:

- Ping dai PC al Web Server: verifichiamo che le macchine comunichino correttamente con il Server
- Verifica dei Verbi HTTP: utilizzando il programma creato in precedenza invieremo richieste HTTP verificandone le risposte
- Utilizzo di un Port Scanner: con il Port Scanner creato in precedenza eseguiremo una scansione delle porte verificandone la sicurezza e l'accessibilità

5.1 Ping PC-Web Server

```
(kali@kali)-[~]
$ ping -w 10 172.16.0.2
PING 172.16.0.2 (172.16.0.2) 56(84) bytes of data:
64 bytes from 172.16.0.2: icmp_seq=1 ttl=63 time=3.97 ms
64 bytes from 172.16.0.2: icmp_seq=2 ttl=63 time=1.41 ms
64 bytes from 172.16.0.2: icmp_seq=3 ttl=63 time=1.51 ms
64 bytes from 172.16.0.2: icmp_seq=4 ttl=63 time=8.43 ms
64 bytes from 172.16.0.2: icmp_seq=5 ttl=63 time=3.89 ms
64 bytes from 172.16.0.2: icmp_seq=6 ttl=63 time=2.98 ms
64 bytes from 172.16.0.2: icmp_seq=7 ttl=63 time=10.6 ms
64 bytes from 172.16.0.2: icmp_seq=8 ttl=63 time=6.73 ms
64 bytes from 172.16.0.2: icmp_seq=9 ttl=63 time=7.35 ms
64 bytes from 172.16.0.2: icmp_seq=10 ttl=63 time=3.99 ms

— 172.16.0.2 ping statistics —
10 packets transmitted, 10 received, 0% packet loss, time 9013ms
rtt min/avg/max/mdev = 1.409/5.084/10.594/2.897 ms
```

Tutti i pacchetti ICMP inviati sono stati ricevuti con successo, funziona tutto correttamente.

5.2 Verifica verbi HTTP

```
(kali@kali)-[~]
$ python3 http_verb.py
Inserisci l'URL (es. http://example.com): http://172.16.0.2
Inserisci il percorso (es. /api/data): /phpMyAdmin/index.php

ANALISI SICUREZZA HTTP
URL BERSAGLIO: http://172.16.0.2/phpMyAdmin/index.php
DATA E ORA: 2026-04-29 07:30:14

[+] GET : ABILITATO (Status: 200)
[+] POST : ABILITATO (Status: 200)
[+] PUT : ABILITATO (Status: 200)
[+] DELETE : ABILITATO (Status: 200)
[+] PATCH : ABILITATO (Status: 200)
[+] HEAD : ABILITATO (Status: 200)
[+] OPTIONS : ABILITATO (Status: 200)
[+] TRACE : ABILITATO (Status: 200)
[-] CONNECT : ERRORE/DISABILITATO (Status: 400)

RIEPILOGO FINALE:
VERBI APERTI: ['GET', 'POST', 'PUT', 'DELETE', 'PATCH', 'HEAD', 'OPTIONS', 'TRACE']
VERBI PROTETTI: []
VERBI CHIUSI: ['CONNECT']
```

Abbiamo, tramite l'utilizzo del nostro codice, inviato richieste di GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS, TRACE e CONNECT.

Il risultato ci mostra che tutti i verbi HTTP sono abilitati tranne CONNECT, perchè il server Apache non è configurato per agire come Proxy.

5.3 Port Scanning

```
(kali@kali)-[~]
$ python3 port_scan.py
Inserisci l'indirizzo IP da scansionare: 172.16.0.2
Inserisci il range di porte da scansionare (es. da 5-200): 1-30

=====
SCANSIONE SU: 172.16.0.2
INIZIO:      2026-04-29 09:18:37
=====
```

PORTA	STATO	SERVIZIO
1	[CHIUSA]	-
2	[CHIUSA]	-
3	[CHIUSA]	-
4	[CHIUSA]	-
5	[CHIUSA]	-
6	[CHIUSA]	-
7	[CHIUSA]	-
8	[CHIUSA]	-
9	[CHIUSA]	-
10	[CHIUSA]	-
11	[CHIUSA]	-
12	[CHIUSA]	-
13	[CHIUSA]	-
14	[CHIUSA]	-
15	[CHIUSA]	-
16	[CHIUSA]	-
17	[CHIUSA]	-
18	[CHIUSA]	-
19	[CHIUSA]	-
20	[CHIUSA]	-
21	[APERTA]	ftp
22	[APERTA]	ssh
23	[APERTA]	telnet
24	[CHIUSA]	-
25	[APERTA]	smtp
26	[CHIUSA]	-
27	[CHIUSA]	-
28	[CHIUSA]	-
29	[CHIUSA]	-
30	[CHIUSA]	-

```
=====
FINE SCANSIONE: 09:18:37
=====
```

Tramite il Port Scanner adesso stiamo verificando l'accessibilità delle porte di comunicazione, possiamo quindi vedere quali servizi sono aperti o chiusi in base al range di porte che abbiamo voluto scansionare.

6.1 Subnetting Rete

La rete della Compagnia Theta è stata progettata con un obiettivo chiaro: dare ad ogni zona esattamente lo spazio che serve, né più né meno, separando le aree in base al loro livello di sicurezza e alla loro funzione.

Rete Internet

Il collegamento verso Internet utilizza una /30, che produce esattamente due indirizzi host utilizzabili. Poiché in quel segmento devono convivere soltanto il gateway del provider e l'interfaccia esterna del firewall, una subnet più grande sarebbe stato puro spreco di indirizzi pubblici, oggi una risorsa scarsa e costosa.

Zona DMZ

La zona DMZ adotta invece una /24, molto più generosa. Attualmente ospita un solo Web Server, ma la DMZ è per definizione la zona destinata a crescere: un domani potrebbero aggiungersi server di posta, portali VPN o altri servizi esposti. Progettarla con ampio margine evita di dover rimettere mano all'indirizzamento ogni volta che l'azienda espande i propri servizi pubblici. Anche la scelta dello spazio 172.16.x.x non è casuale: tenendolo separato dal 192.168.x.x della LAN interna, il traffico DMZ è immediatamente riconoscibile nei log e nelle regole firewall.

LAN

Per la LAN di gestione si è scelto lo spazio 10.0.0.0, tradizionalmente riservato all'infrastruttura di rete. Questo permette di scrivere regole di accesso molto semplici — "solo gli amministratori raggiungono gli indirizzi 10.x.x.x" — senza dover elencare ogni singolo dispositivo.

La scelta più ragionata riguarda però i sei piani aziendali, tutti con maschera /26. Ogni piano ospita 20 PC, e una /26 offre 62 indirizzi utilizzabili: i 20 occupati dai PC lasciano altri 42 liberi per stampanti, telefoni VoIP, dispositivi mobili e qualsiasi periferica futura. Ma il vantaggio principale non è questo. Avere una subnet separata per ogni piano significa che ogni piano è un dominio di broadcast indipendente: il traffico generato dai PC di un piano non raggiunge mai gli altri, riducendo il carico di rete e impedendo che un dispositivo compromesso possa intercettare comunicazioni di altri reparti. Con un'unica grande subnet per tutti i 120 PC, questi vantaggi sarebbero andati completamente perduti.

Infine, distribuire i piani su due ottetti distinti — 192.168.1.x per i piani 0, 1 e 2 e 192.168.2.x per i piani 3, 4 e 5 — risponde a un criterio di leggibilità: guardando un indirizzo IP si capisce immediatamente a quale piano appartiene un dispositivo, rendendo la manutenzione più rapida e gli errori meno probabili.

Tabella completa Subnetting

Zona/Piano	Rete	CIDR	Subnet Mask	Gateway	Range Host	Host Disp.	Note
WAN	200.0.0.0	/30	255.255.255.252	—	200.0.0.1 – 200.0.0.2	2	Internet
DMZ VPN	172.16.0.0	/24	255.255.255.0	172.16.0.1	172.16.0.2 – 172.16.0.254	253	Metasploitable
LAN SW	10.0.0.0	/8	255.0.0.0	10.0.0.1	10.0.0.1 – 10.255.255.254	253	20PC + NAS + IDS
Piano 0	192.168.1.0	/26	255.255.255.192	192.168.1.1	192.168.1.1 – 192.168.1.62	62	20 PC
Piano 1	192.168.1.64	/26	255.255.255.192	192.168.1.45	192.168.1.46 – 192.168.1.109	62	20 PC
Piano 2	192.168.1.128	/26	255.255.255.192	192.168.1.129	192.168.1.130 – 192.168.1.190	62	20 PC
Piano 3	192.168.2.0	/26	255.255.255.192	192.168.2.1	192.168.2.2 – 192.168.2.62	62	20 PC
Piano 4	192.168.2.64	/26	255.255.255.192	192.168.2.65	192.168.2.46 – 192.168.2.109	62	20 PC
Piano 5	192.168.2.128	/26	255.255.255.192	192.168.2.129	192.168.2.130 – 192.168.2.190	62	20 PC

6.2 Socket Sniffer

Il terzo programma ha un approccio completamente diverso dai due precedenti: invece di generare traffico verso un bersaglio, si mette in ascolto passivo sulla scheda di rete e cattura i pacchetti che transitano, mostrando in tempo reale chi comunica con chi e con quale protocollo. Questo tipo di analisi corrisponde, in scala ridotta, al tipo di monitoraggio che nella topologia reale è affidato ai tre IDS Juniper idp250: osservare il traffico che scorre sulla rete per identificare comportamenti anomali.

Scelta degli strumenti

Questo programma usa tre librerie: socket, struct e datetime. Rispetto allo scanner di porte, dove socket veniva usato per aprire connessioni TCP standard, qui viene utilizzato in modalità raw, ovvero per ricevere i pacchetti così come arrivano dalla scheda di rete prima che il sistema operativo li elabori. Questo richiede privilegi di amministratore, ma è l'unico modo per accedere al traffico a questo livello.

La libreria struct è necessaria perché i pacchetti di rete arrivano come sequenze di byte grezzi senza nessuna struttura leggibile. Per estrarre informazioni utili come gli indirizzi IP e il tipo di protocollo bisogna conoscere la struttura esatta dell'header IP e sapere in quale posizione si trova ogni campo. Struct permette di fare questa operazione dichiarando il formato dei dati atteso, senza dover gestire manualmente ogni singolo byte. Senza questa libreria il codice sarebbe molto più lungo e soggetto a errori.

Approccio adottato e motivazioni

La scelta di lavorare direttamente a livello di pacchetti IP, saltando i byte iniziali dell'header Ethernet che non ci interessavano, rispecchia l'obiettivo del programma: osservare il traffico IP indipendentemente dall'interfaccia fisica su cui transita. Filtrare solo i pacchetti IPv4 fin dall'apertura del socket era importante per evitare di elaborare altri tipi di frame che non avrebbero contribuito all'analisi. Abbiamo incluso un dizionario di protocolli per tradurre i numeri in nomi leggibili, rendendo l'output immediatamente interpretabile senza dover consultare riferimenti esterni.

Codice Socket-Sniffer

```
GNU nano 8.7.1 socket_sniffer.py
import socket
import struct
from datetime import datetime
def nome_protocollo(proto):
    protocoll = {
        1: "ICMP",
        6: "TCP",
        17: "UDP",
        2: "IGMP",
    }
    return protocoll.get(proto, f"ALTRO({proto})")
def analizza_ip(raw):
    iph = struct.unpack('!BBHHBBH4s4s',
        raw[:20])
    src = socket.inet_ntoa(iph[8])
    dst = socket.inet_ntoa(iph[9])
    proto = iph[6]
    return src, dst, proto
def socket_sniffer():
    print(f"\n{'='*55}")
    print(f"SOCKET SNIFFER LINUX")
    print(f"AVVIATO: {datetime.now().strftime('%d/%m/%Y %H:%M:%S')}")
    print(f"Premi CTRL + C per fermare")
    print(f"\n{'='*55}")
    s = socket.socket(
        socket.AF_PACKET,
        socket.SOCK_RAW,
        socket.htons(0x0800)
    )
    contatore = 0
    try:
        while True:
            raw_data, addr = s.recvfrom(65535)
            raw_ip = raw_data[14:]
            if len(raw_ip) < 20:
                continue
            src, dst, proto = analizza_ip(raw_ip)
            nome_proto = nome_protocollo(proto)
            contatore += 1
            ora = datetime.now().strftime('%H:%M:%S')
            print(f" [{ora}] #{contatore:04d} "
                f"{nome_proto:6}| "
                f"{src:20} {dst:20}")
    except KeyboardInterrupt:
        print(f"\n{'='*55}")
        print(f"SNIFFING FERMATO!")
        print(f"PACCHETTI CATTURATI: {contatore}")
        print(f"\n{'='*55}")
    finally:
        s.close()
if __name__ == "__main__":
    socket_sniffer()
```


7 Conclusioni

Azienda Theta



Spendiamo ora due parole sul progetto, su come ci siamo suddivisi il lavoro, le sfide affrontate e le soluzioni e come ci siamo trovati ad affrontare questa prima Build Week.

Da subito c'è stata molta sinergia nel gruppo, ognuno ha contribuito a seconda delle proprie skill facendo sempre presente agli altri membri del gruppo il lavoro svolto, in maniera tale che tutti capissero esattamente cosa si stesse facendo e perchè.

7.1 Suddivisione del lavoro

Tommaso Tarullo

- Topologia
- Configurazione di rete
- Guida agli script Python
- Reportistica descrittiva Topologia
- Subnetting

Simone Tesoro

- Script HTTP verbs
- Script Port Scanner
- Virtualizzazione
- Regole Firewall

Ginaluca Vitale

- Reportistica Virtualizzazione
- Reportistica Python descrittiva
- Reportistica Testing e Ping

Nicholas Spigolon

- Reportistica Tecnica Topologia
- Preventivo
- Reportistica Preventivo

Tancredi De Simone

- Preventivo
- Reportistica Preventivo
- Formattazione e impaginazione Report
- Reportistica tecnica Topologia

Tutti

- Socket Sniffer

7.2 Considerazioni Finali

Sebbene ci sia stata una suddivisione del lavoro, facciamo presente che visto che ognuno di noi voleva assorbire completamente ogni informazione e avere presente per bene tutte le fasi del progetto, in realtà in ogni passaggio ognuno ha dato il suo contributo.

Ci siamo trovati molto bene in questa prima Build Week, il team era affiatato, ci siamo molto divertiti e abbiamo imparato tantissimo. Tommaso è stato un punto di riferimento fondamentale per il gruppo e ha fatto in modo che tutti imparassimo e capissimo perfettamente ogni passaggio.

